HACKTHEBOX

Loggy Writeup

10th June 2024

Prepared by: WibblyWobbly + VivisGhost

Malware Author: WibblyWobbly

Difficulty: Easy

Scenario

> Janice from accounting is beside herself! She was contacted by the SOC to tell her that her work credentials were found on the dark web by the threat intel team. They investigated and scanned her machine but couldn't find any malicious activity. She changed her password anyway as a precaution, but she was contacted by the SOC that her credentials were shared on the dark web again! This time, during the investigation, it was discovered that she was using a screenshot tool she found on a sketchy site. The forensic team managed to recover the suspicious executable and some files and gave it to their malware reverse engineer to analyze.

Artifacts Provided

- Loggy.zip - *4226e5f90a5dc89807d001986ae78f00e2549dbb7c57d6165e93c9478de8b3e6*
  - Loggy.exe - *6acd8a362def62034cbd011e6632ba5120196e2011c83dc6045fcb28b590457c*
  - keylog.txt - *7dfb5222834c48e00ad25e66a521ca13c8dd4ee4bd2aeee06ff9c20ea40b238d*
  - screenshot_1718068575.png - *54c190d80e6c2c8f41ada58c293f0cd6a509710a8d6de22e77c0b20055ceef2f*
  - screenshot_1718068581.png - *f0fbd9cf8bb1b068067419d1fd8c454b45d9d11ac574e731c00c95366b45a86d*
  - screenshot_1718068593.png - *8f531c1511572e5e6af70a1c822e4d0659f8cf8c5e6e84e63066559eb57c3102*
  - screenshot_1718068600.png - *7e29f8d7c4495e91559108d2bf004f8ba20046b47015ec0d2a98a73d408cd8a1*

```
┌─[vivisghost@HackTheBox]─[~/Loggy]
└──╼ $sha256sum Loggy.zip
4226e5f90a5dc89807d001986ae78f00e2549dbb7c57d6165e93c9478de8b3e6  Loggy.zip
```

Skills Learnt

- Malware Analysis

Tags

- Malware Analysis
- Golang

Initial Analysis

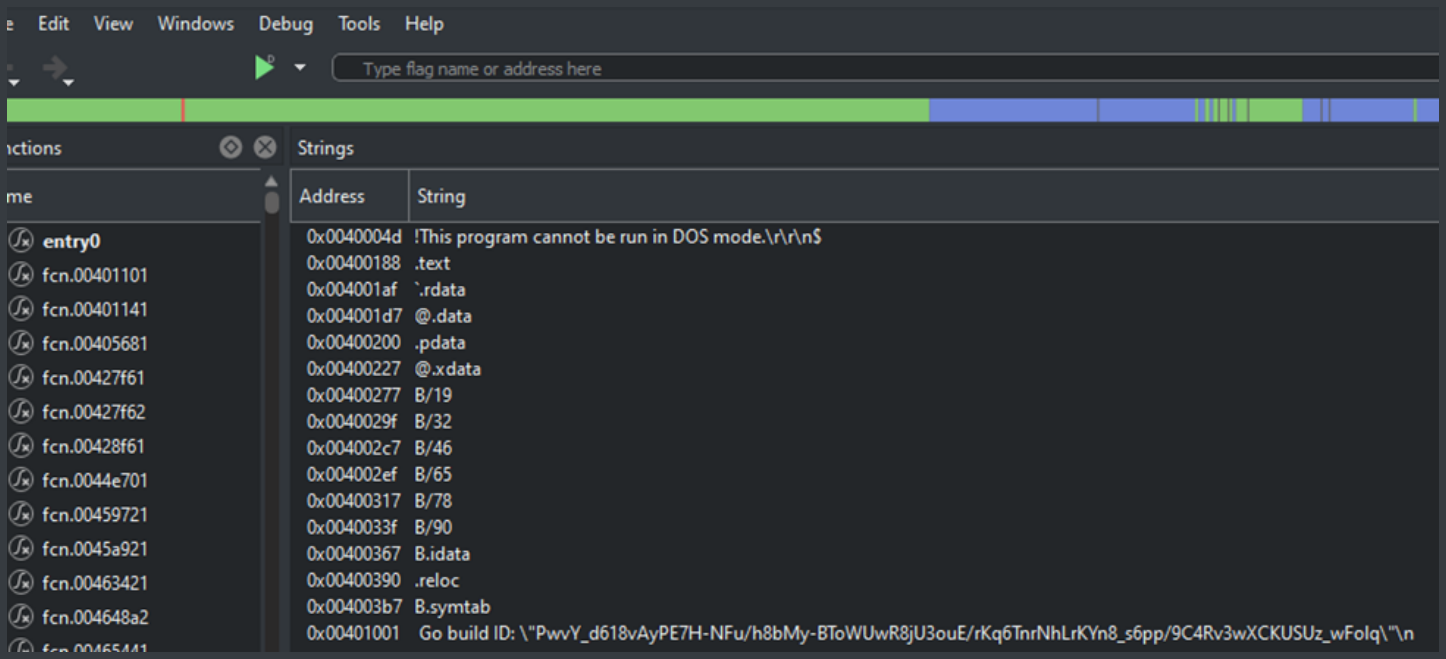
Since this is malware analysis, this shall be done entirely in a set of VMs: FlareVM for Windows binaries and REMnux to act as my mock DNS server. They are isolated from the internet or my host machine.

To begin the analysis, the password-protected ZIP file was unlocked using the password `hacktheblue`. Next, let's check what we have been given.

```
┌─[✗]─[vivisghost@HackTheBox]─[~/Loggy]
└──╼ $unzip -l Loggy.zip
Archive:  Loggy.zip
  Length      Date    Time    Name
---------  ---------- -----   ----
     1050  2024-12-06 11:05   DANGER.txt
  3781456  2024-12-06 11:07   danger.zip
     7111  2024-06-09 19:01   keylog.txt
  1316494  2024-06-10 16:16   screenshot_1718068575.png
   928503  2024-06-10 16:16   screenshot_1718068581.png
  1515653  2024-06-10 16:16   screenshot_1718068593.png
  1515105  2024-06-10 16:16   screenshot_1718068600.png
---------                     -------
  9065372                     7 files
```

Next, we will need to use the password found in `DANGER.txt` to open `danger.zip`.

There are multiple ways to get the static strings, here I show Cutter.



Even with this short snippet we get a clue it might be a Go binary.

Questions

1. **What is the SHA-256 hash of this malware binary?**

   We can use the following command to get the hash.

   `sha256sum Loggy.exe`

   Answer: `6acd8a362def62034cbd011e6632ba5120196e2011c83dc6045fcb28b590457c`

2. **What programming language (and version) is this malware written in?**

   In the initial analysis we were able to see it was a Go binary. We can use `go version -m` to get its version and dependencies.

```
┌─[vivisghost@HackTheBox]─[~/Loggy/Loggy_Artifacts]
└──╼ $go version -m Loggy.exe
Loggy.exe: go1.22.3
        path    command-line-arguments
        dep     github.com/TheTitanrain/w32     v0.0.0-20200114052255-2654d97dbd3d      h1:2xp1BQbqcDDaikHnASWpVZRjibOxu7y9LhAv04whugI=
        dep     github.com/hashicorp/errwrap    v1.0.0  h1:hLrqtEDnRye3+sgx6z4qVLNuviH3MR5aQ0ykNJa/UYA=
        dep     github.com/hashicorp/go-multierror      v1.1.1  h1:H5DkEtf6CXdFp0N0Em5UCwQpXMWke8IA0+lD48awMYo=
        dep     github.com/jlaffaye/ftp v0.2.0  h1:lXNvW7cBu7R/68bknOX3MrRIIqZ61zELs1P2RAiA3lg=
        dep     github.com/kbinani/screenshot   v0.0.0-20230812210009-b87d31814237      h1:YOp8St+CM/AQ9Vp4XYm4272E77MptJDHkwypQHIRl9Q=
        dep     github.com/lxn/win      v0.0.0-20210218163916-a377121e959e      h1:H+t6A/QJMbhCSEH5rAuRxh+CtW96g0Or0Fxa9IKr4uc=
        dep     golang.org/x/sys        v0.11.0 h1:eG7RXZHdqOJ1i+0lgLgCpSXAp6M3LYlAo6osgSi0xOM=
        build   -buildmode=exe
        build   -compiler=gc
        build   CGO_ENABLED=1
        build   CGO_CFLAGS=
        build   CGO_CPPFLAGS=
        build   CGO_CXXFLAGS=
        build   CGO_LDFLAGS=
        build   GOARCH=amd64
        build   GOOS=windows
        build   GOAMD64=v1
```

   We can see the version, dependencies and build; exe, Windows OS and amd64 architecture.

   Answer: `Golang 1.22.3`

3. **There are multiple GitHub repos referenced in the static strings. Which GitHub repo would be most likely suggest the ability of this malware to exfiltrate data?**

Building upon the initial analysis, with static strings I typically CTRL+F with common top-level domains (.com, .net, .ru, .cn), the "github" string, and common internet protocols (http, https, ftp, sftp). This produces fruitful results:

```
github.com/jlaffaye/ftp.(*ServerConn).Login
github.com/jlaffaye/ftp.(*ServerConn).Type
github.com/jlaffaye/ftp.(*ServerConn).feat
```

This can also be found in the `version` command from question #2.

Answer: `github.com/jlaffaye/ftp`

4. **What dependency, expressed as a GitHub repo, supports Janice's assertion that she thought she downloaded something that can just take screenshots?**

Also, I see a reason to believe that Janice thought she was downloading a program designed to take screenshots:

```
path    command-line-arguments
dep     github.com/TheTitanrain/w32     v0.0.
dep     github.com/hashicorp/errwrap    v1.0.
dep     github.com/hashicorp/go-multierror
dep     github.com/jlaffaye/ftp v0.2.0  h1:lX
dep     github.com/kbinani/screenshot   v0.0.
dep     github.com/lxn/win      v0.0.0-202102
dep     golang.org/x/sys        v0.11.0 h1:eG
```

This can also be found in the `version` command from question #2.

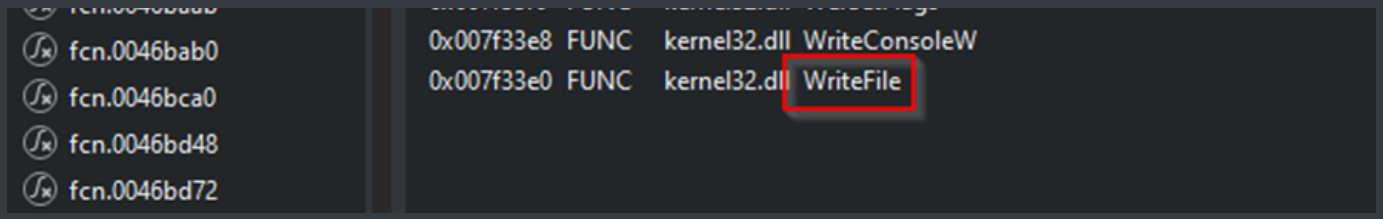
Answer: `github.com/kbinani/screenshot`

5. **Which function call suggests that the malware produces a file after execution?**

To get an idea of the malware's functionality, I check the imports. Interesting to note is WriteFile, which suggests that this malware writes something to disk:

ame
entry0
fcn.00401101
fcn.00401141
fcn.00405681
fcn.00427f61
fcn.00427f62
fcn.00428f61
fcn.0044e701
fcn.00459721
fcn.0045a921
fcn.00463421
fcn.004648a2
fcn.00465441
fcn.004659e1
fcn.004659e2
fcn.004667c1
fcn.004667e1
fcn.00466801
fcn.00466821
fcn.0046b3a1
fcn.0046b9a9
fcn.0046b9f8
fcn.0046ba01
fcn.0046ba0f
fcn.0046ba18
fcn.0046ba3d
fcn.0046ba46
fcn.0046ba5d
fcn.0046ba6b
fcn.0046ba74
fcn.0046ba78
fcn.0046ba82
fcn.0046ba8b
fcn.0046ba8f
fcn.0046ba94
fcn.0046ba99
fcn.0046baa2
fcn.0046baa6
fcn.0046baab

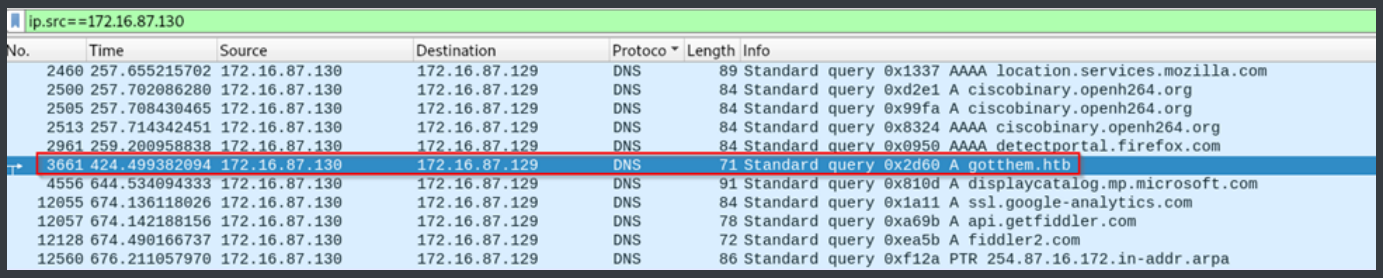
Address	Type	Library	Name	Safety
0x007f3550	FUNC	kernel32.dll	AddVectoredContinueHandler	
0x007f3548	FUNC	kernel32.dll	AddVectoredExceptionHandler	
0x007f3540	FUNC	kernel32.dll	CloseHandle	
0x007f3538	FUNC	kernel32.dll	CreateEventA	
0x007f3530	FUNC	kernel32.dll	CreateFileA	
0x007f3528	FUNC	kernel32.dll	CreateIoCompletionPort	
0x007f3520	FUNC	kernel32.dll	CreateThread	
0x007f3518	FUNC	kernel32.dll	CreateWaitableTimerExW	
0x007f3510	FUNC	kernel32.dll	DuplicateHandle	
0x007f3508	FUNC	kernel32.dll	ExitProcess	
0x007f3500	FUNC	kernel32.dll	FreeEnvironmentStringsW	
0x007f34f8	FUNC	kernel32.dll	GetConsoleMode	
0x007f34f0	FUNC	kernel32.dll	GetCurrentThreadId	
0x007f34e8	FUNC	kernel32.dll	GetEnvironmentStringsW	
0x007f34e0	FUNC	kernel32.dll	GetErrorMode	
0x007f34d8	FUNC	kernel32.dll	GetProcAddress	
0x007f34d0	FUNC	kernel32.dll	GetProcessAffinityMask	
0x007f34c8	FUNC	kernel32.dll	GetQueuedCompletionStatusEx	
0x007f34c0	FUNC	kernel32.dll	GetStdHandle	
0x007f34b8	FUNC	kernel32.dll	GetSystemDirectoryA	
0x007f34b0	FUNC	kernel32.dll	GetSystemInfo	
0x007f34a8	FUNC	kernel32.dll	GetThreadContext	
0x007f3498	FUNC	kernel32.dll	LoadLibraryExW	
0x007f3490	FUNC	kernel32.dll	LoadLibraryW	
0x007f3488	FUNC	kernel32.dll	PostQueuedCompletionStatus	
0x007f3480	FUNC	kernel32.dll	RaiseFailFastException	
0x007f3478	FUNC	kernel32.dll	ResumeThread	
0x007f3470	FUNC	kernel32.dll	RtlLookupFunctionEntry	
0x007f3468	FUNC	kernel32.dll	RtlVirtualUnwind	
0x007f3460	FUNC	kernel32.dll	SetConsoleCtrlHandler	
0x007f3458	FUNC	kernel32.dll	SetErrorMode	
0x007f3450	FUNC	kernel32.dll	SetEvent	
0x007f3448	FUNC	kernel32.dll	SetProcessPriorityBoost	
0x007f34a0	FUNC	kernel32.dll	SetThreadContext	
0x007f3440	FUNC	kernel32.dll	SetWaitableTimer	
0x007f3438	FUNC	kernel32.dll	SuspendThread	
0x007f3430	FUNC	kernel32.dll	SwitchToThread	
0x007f3428	FUNC	kernel32.dll	TlsAlloc	
0x007f3420	FUNC	kernel32.dll	VirtualAlloc	
0x007f3418	FUNC	kernel32.dll	VirtualFree	
0x007f3410	FUNC	kernel32.dll	VirtualQuery	
0x007f3400	FUNC	kernel32.dll	WaitForMultipleObjects	
0x007f3408	FUNC	kernel32.dll	WaitForSingleObject	
0x007f33f8	FUNC	kernel32.dll	WerGetFlags	
0x007f33f0	FUNC	kernel32.dll	WerSetFlags	



Answer: `WriteFile`

6. **You observe that the malware is exfiltrating data over FTP. What is the domain it is exfiltrating data to?**

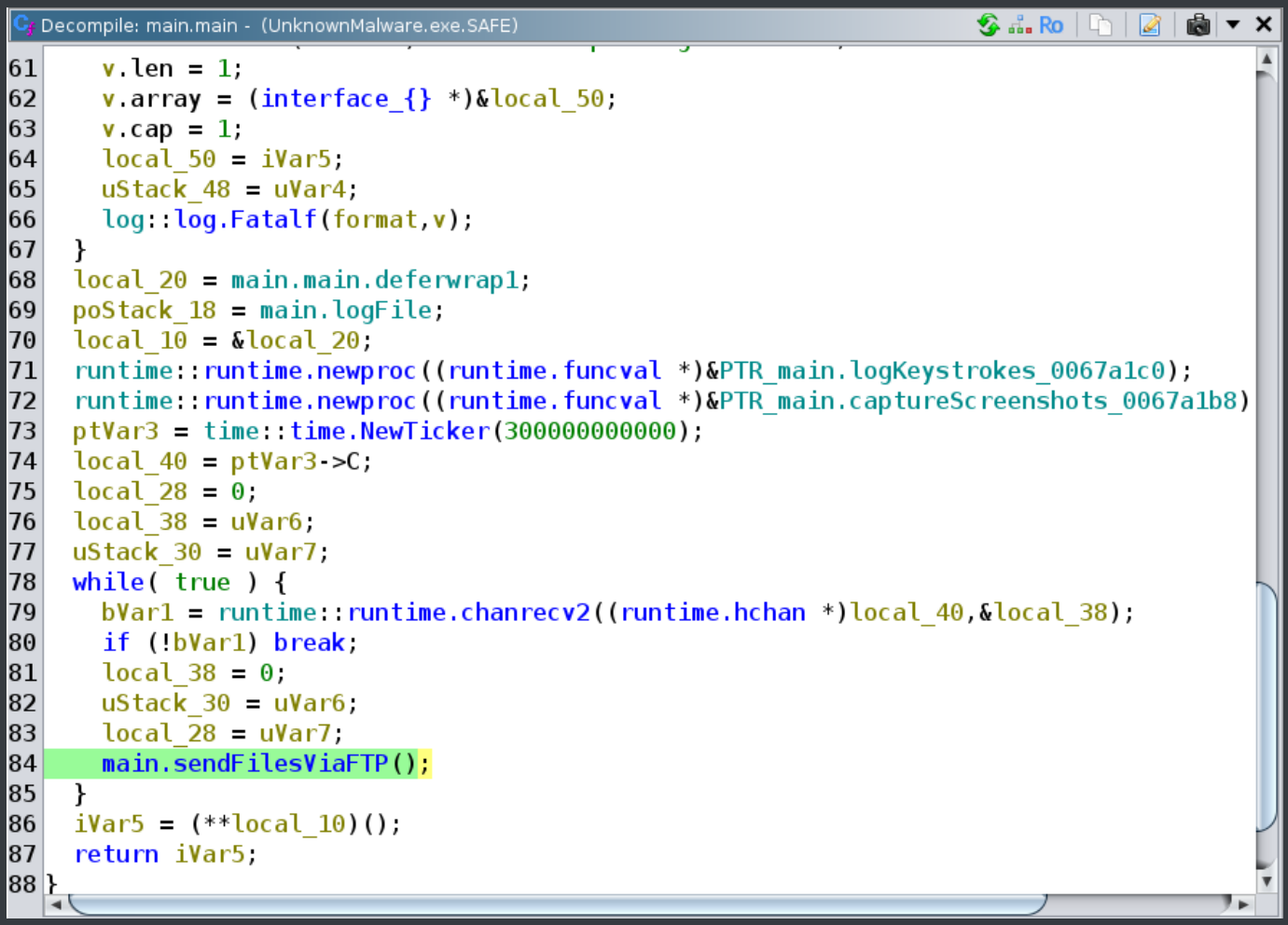
   For dynamic analysis, I will finally detonate the malware. After detonation I immediately check Wireshark in my REMnux VM.
   I see an interesting DNS query:



   We can also find this with static analysis.

   With Go binaries, the main functionality of the program is often in `main.main`. Opening the executable in Ghidra and navigating to `main.main` a couple functions stick out.

   - `logKeystokes` - line 71
   - `captureScreenshots` - line 72
   - `sendFilesViaFTP()` - line 84



Viewing the `sendFilesViaFTP()` function we can see the address being used in the FTP connection on line 77.

Answer: `gotthem.htb`

7. **What are the threat actor's credentials?**

Continuing with the dynamic analysis and considering the hints at FTP functionality in the static strings, I look for FTP requests within Wireshark, and I find:

ip.src==172.16.87.130

No.	Time	Source	Destination	Protocol	Length	Info
3667	424.520562905	172.16.87.130	172.16.87.129	FTP	72	Request: USER NottaHacker
3670	424.521624634	172.16.87.130	172.16.87.129	FTP	78	Request: PASS Cle@rtextP@ssword
3673	424.522578030	172.16.87.130	172.16.87.129	FTP	60	Request: FEAT
3676	424.523780964	172.16.87.130	172.16.87.129	FTP	62	Request: TYPE I
3679	424.524997763	172.16.87.130	172.16.87.129	FTP	60	Request: EPSV
3682	424.525630278	172.16.87.130	172.16.87.129	FTP	60	Request: PASV

We have threat actor credentials!

This could also be found with static analysis in the same function as shown in the previous question. Here we can see the password on line 108 and the username on line 110.

```
107  password.len = 0x11;
108  password.str = (uint8 *)"Cle@rtextP@ssword";
109  user.len = 0xb;
110  user.str = (uint8 *)"NottaHacker";
111  eVar2 = github.com/jlaffaye/ftp::github.com/jlaffaye/ftp.(*ServerConn).Login
112                    (local_d8,user,password);
113  pvStack_78 = eVar2.data;
114  local_80 = (internal/abi.Type *)eVar2.tab;
```

Answer: `NottaHacker:Cle@rtextP@ssword`

8. **What file keeps getting written to disk?**

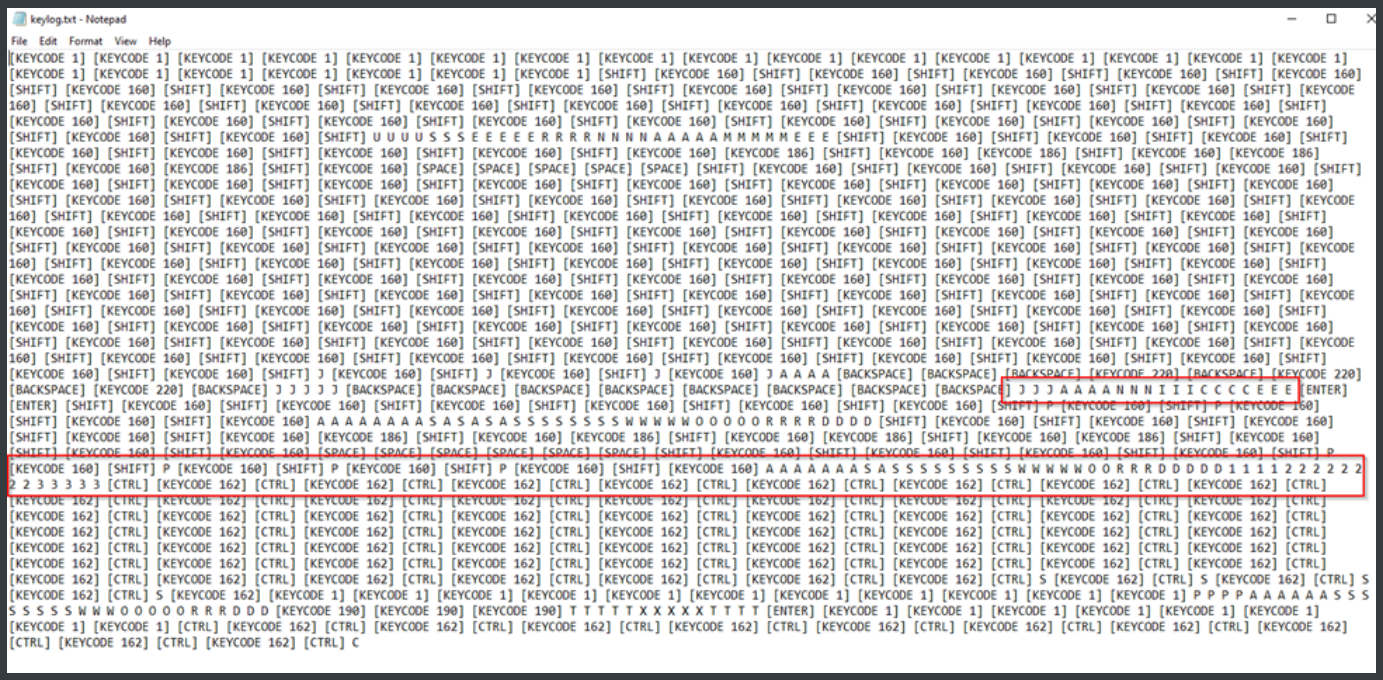
When I look at Process Monitor, I see that a single file keeps getting written to disk:

4872	WriteFile	C:\Users[redacted]\keylog.txt	SUCCESS	Offset: -1, Length: ...
4872	FlushBuffersFile	C:\Users[redacted]\keylog.txt	SUCCESS	
4872	WriteFile	C:\Users[redacted]\keylog.txt	SUCCESS	Offset: 4,096, Leng...
4872	WriteFile	C:\Users[redacted]\keylog.txt	SUCCESS	Offset: -1, Length: 12
4872	FlushBuffersFile	C:\Users[redacted]\keylog.txt	SUCCESS	
4872	WriteFile	C:\Users[redacted]\keylog.txt	SUCCESS	Offset: 4,096, Leng...
4872	WriteFile	C:\Users[redacted]\keylog.txt	SUCCESS	Offset: -1, Length: 12
4872	FlushBuffersFile	C:\Users[redacted]\keylog.txt	SUCCESS	
4872	WriteFile	C:\Users[redacted]\keylog.txt	SUCCESS	Offset: 4,096, Leng...
4872	WriteFile	C:\Users[redacted]\keylog.txt	SUCCESS	Offset: -1, Length: 12
4872	FlushBuffersFile	C:\Users[redacted]\keylog.txt	SUCCESS	
4872	WriteFile	C:\Users[redacted]\keylog.txt	SUCCESS	Offset: 4,096, Leng...
4872	WriteFile	C:\Users[redacted]\keylog.txt	SUCCESS	Offset: -1, Length: 12
4872	FlushBuffersFile	C:\Users[redacted]\keylog.txt	SUCCESS	
4872	WriteFile	C:\Users[redacted]\keylog.txt	SUCCESS	Offset: 4,096, Leng...

Answer: `keylog.txt`

9. **When Janice changed her password, this was captured in a file. What is Janice's username and password?**

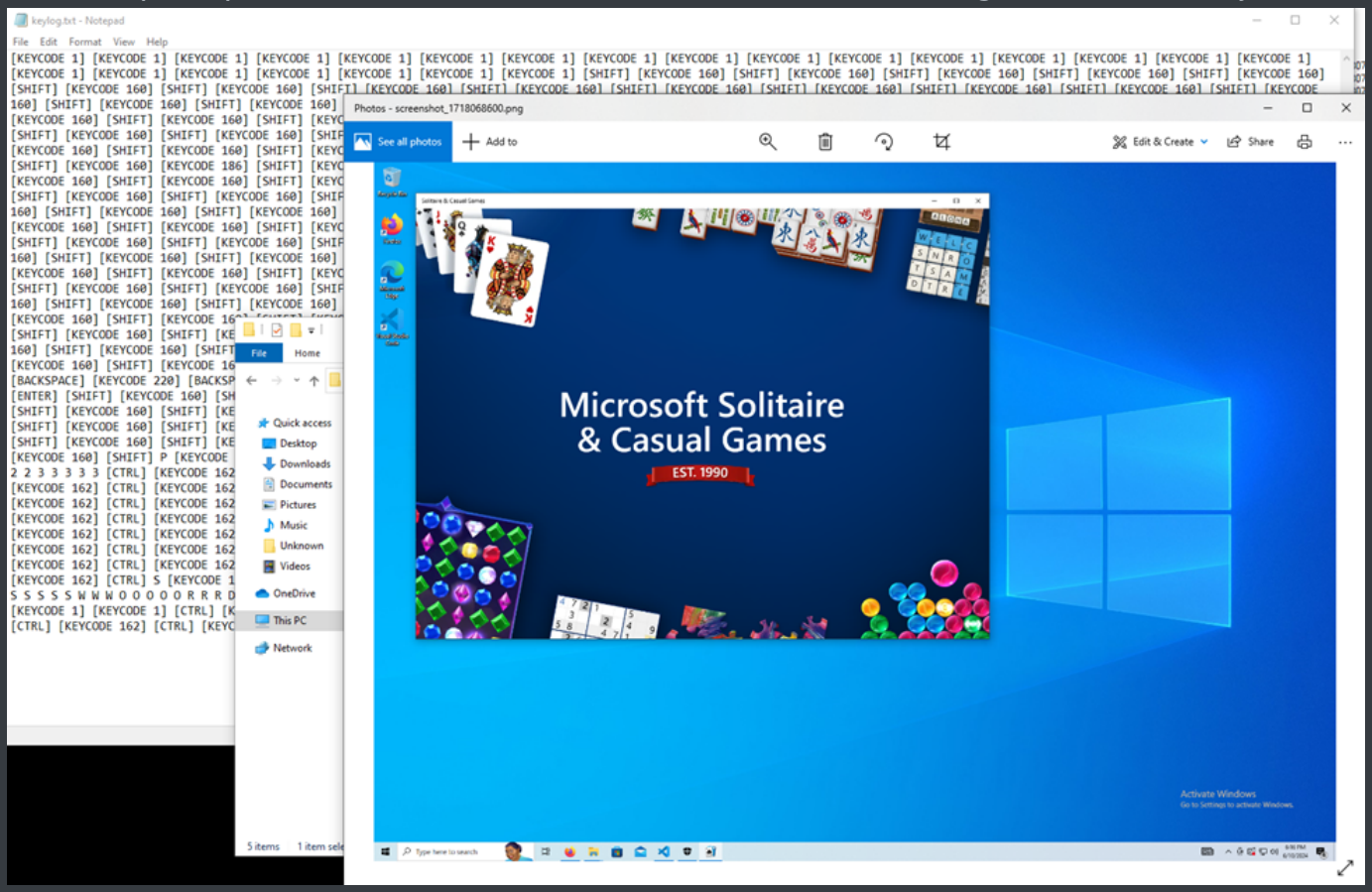
   When I open up the keylog.txt file, I can see that Janice entered her username and password. The first highlight shows that her username was entered with all lowercase characters. The second highlight shows that Janice entered her password with an uppercase "P" due to the use of the SHIFT key right before entering it:



   Answer: `janice:Password123`

10. **What app did Janice have open the last time she ran the "screenshot app"?**

When I open up the screenshots, I can see what Janice is working on while on the job:



Answer: `Solitaire`

There you have it! We now know why Janice's credentials keep getting leaked. She was running a keylogger from some sketchy website. It was sloppy work and even kept the keylog file and screenshots in the same directory as the executable. Maybe if she wasn't too busy playing Solitaire she might have realized something was amiss before the threat intel team found her creds on the dark web!